

DBMS WEEK 4 ASSIGNMENT

Task Name: Order Management System

1. Introduction

An Order Management System is a database application used to manage customer orders and the products included in those orders. The system stores information about customers, products, orders, and order details.

The main purpose of this system is to efficiently record customer orders, maintain product quantities, calculate total order amounts, modify existing orders, and generate customer order history reports.

This project demonstrates the use of relational database concepts such as tables, primary keys, foreign keys, constraints, SQL queries, insertion, modification, and reporting.

2. Objectives

The main objectives of the Order Management System are:

1. To design and create Orders and Order_Details tables.
 2. To manage customer product orders.
 3. To store order date, product quantity, and total amount.
 4. To perform order insertion operations.
 5. To perform modification and update operations on orders.
 6. To maintain relationships between customers, orders, and products.
 7. To generate customer order history reports.
 8. To demonstrate the practical use of SQL in a DBMS.
-

3. Database Design

The Order Management System consists of the following main tables:

- Customers – stores customer information.
- Products – stores product information.
- Orders – stores information about customer orders.
- Order_Details – stores the products and quantities included in each order.

Relationship

The relationships between the tables are:

- One customer can place many orders.
- One order can contain many products.

- One product can appear in many orders.
- Order_Details connects the Orders and Products tables.

The relationship can be represented as:

Customers → Orders → Order_Details ← Products

4. Table Structure

Customers Table

Field	Data Type	Description
Customer_ID	INT	Primary key
Customer_Name	VARCHAR(100)	Customer name
Email	VARCHAR(100)	Customer email
Phone	VARCHAR(15)	Customer phone number

Products Table

Field	Data Type	Description
Product_ID	INT	Primary key
Product_Name	VARCHAR(100)	Product name
Price	DECIMAL(10,2)	Product price
Stock	INT	Available quantity

Orders Table

Field	Data Type	Description
Order_ID	INT	Primary key
Customer_ID	INT	Foreign key
Order_Date	DATE	Date of order
Total_Amount	DECIMAL(10,2)	Total order amount

Order_Details Table

Field	Data Type	Description
Order_Detail_ID	INT	Primary key
Order_ID	INT	Foreign key

Field	Data Type	Description
Product_ID	INT	Foreign key
Quantity	INT	Quantity ordered
Unit_Price	DECIMAL(10,2)	Price per product
Subtotal	DECIMAL(10,2)	Quantity × Unit Price

5. SQL Table Creation

Create Database

```
CREATE DATABASE OrderManagementDB;
```

```
USE OrderManagementDB;
```

Create Customers Table

```
CREATE TABLE Customers (
    Customer_ID INT PRIMARY KEY,
    Customer_Name VARCHAR(100) NOT NULL,
    Email VARCHAR(100) UNIQUE,
    Phone VARCHAR(15)
);
```

Create Products Table

```
CREATE TABLE Products (
    Product_ID INT PRIMARY KEY,
    Product_Name VARCHAR(100) NOT NULL,
    Price DECIMAL(10,2) NOT NULL,
    Stock INT NOT NULL
);
```

Create Orders Table

```
CREATE TABLE Orders (
    Order_ID INT PRIMARY KEY,
    Customer_ID INT NOT NULL,
    Order_Date DATE NOT NULL,
```

```
Total_Amount DECIMAL(10,2) DEFAULT 0,  
FOREIGN KEY (Customer_ID) REFERENCES Customers(Customer_ID)  
);
```

Create Order_Details Table

```
CREATE TABLE Order_Details (  
    Order_Detail_ID INT PRIMARY KEY,  
    Order_ID INT NOT NULL,  
    Product_ID INT NOT NULL,  
    Quantity INT NOT NULL,  
    Unit_Price DECIMAL(10,2) NOT NULL,  
    Subtotal DECIMAL(10,2) NOT NULL,  
    FOREIGN KEY (Order_ID) REFERENCES Orders(Order_ID),  
    FOREIGN KEY (Product_ID) REFERENCES Products(Product_ID)  
);
```

6. Insert Sample Customer Data

```
INSERT INTO Customers  
(Customer_ID, Customer_Name, Email, Phone)  
VALUES  
(1, 'Rahul Kumar', 'rahul@gmail.com', '9876543210'),  
(2, 'Priya Sharma', 'priya@gmail.com', '9876543211'),  
(3, 'Arun Kumar', 'arun@gmail.com', '9876543212'),  
(4, 'Sneha Patel', 'sneha@gmail.com', '9876543213');
```

7. Insert Sample Product Data

```
INSERT INTO Products  
(Product_ID, Product_Name, Price, Stock)  
VALUES  
(101, 'Laptop', 55000.00, 20),  
(102, 'Keyboard', 1500.00, 50),  
(103, 'Mouse', 800.00, 75),
```

(104, 'Monitor', 12000.00, 30),
(105, 'Headphones', 2500.00, 40);

8. Order Insertion Operation

Suppose customer Rahul Kumar purchases one laptop and one mouse.

First, insert the order into the Orders table:

```
INSERT INTO Orders
```

```
(Order_ID, Customer_ID, Order_Date, Total_Amount)
```

```
VALUES
```

```
(1001, 1, '2026-08-15', 55800.00);
```

Then insert the products into the Order_Details table:

```
INSERT INTO Order_Details
```

```
(Order_Detail_ID, Order_ID, Product_ID, Quantity, Unit_Price, Subtotal)
```

```
VALUES
```

```
(1, 1001, 101, 1, 55000.00, 55000.00),
```

```
(2, 1001, 103, 1, 800.00, 800.00);
```

The total amount is:

55000 + 800 = 55800

9. Insert Another Order

Suppose Priya Sharma purchases two keyboards and one headphone.

```
INSERT INTO Orders
```

```
(Order_ID, Customer_ID, Order_Date, Total_Amount)
```

```
VALUES
```

```
(1002, 2, '2026-08-15', 5500.00);
```

Insert the order details:

```
INSERT INTO Order_Details
```

```
(Order_Detail_ID, Order_ID, Product_ID, Quantity, Unit_Price, Subtotal)
```

```
VALUES
```

```
(3, 1002, 102, 2, 1500.00, 3000.00),
```

```
(4, 1002, 105, 1, 2500.00, 2500.00);
```

Total amount:

$$3000 + 2500 = 5500$$

10. Order Modification Operation

If the customer changes the quantity of a product, the order details can be updated.

For example, if Rahul wants two mice instead of one:

```
UPDATE Order_Details
```

```
SET Quantity = 2,
```

```
    Subtotal = 1600.00
```

```
WHERE Order_Detail_ID = 2;
```

The total order amount should also be updated:

```
UPDATE Orders
```

```
SET Total_Amount = 56600.00
```

```
WHERE Order_ID = 1001;
```

The new total is:

$$55000 + 1600 = 56600$$

11. Updating Product Price

Product prices can also be modified when required.

For example:

```
UPDATE Products
```

```
SET Price = 1600.00
```

```
WHERE Product_ID = 102;
```

This updates the price of the keyboard from ₹1500 to ₹1600.

12. Deleting an Order Detail

If a product needs to be removed from an order, the corresponding order detail can be deleted.

```
DELETE FROM Order_Details
```

```
WHERE Order_Detail_ID = 4;
```

After deleting an order detail, the total amount of the order should be recalculated.

13. Display All Orders

The following query displays all orders:

```
SELECT *  
FROM Orders;
```

14. Display Order Details

To display all products included in orders:

```
SELECT *  
FROM Order_Details;
```

15. Customer Order History Report

A customer order history report shows the customer name, order ID, order date, product name, quantity, and total amount.

```
SELECT  
    c.Customer_Name,  
    o.Order_ID,  
    o.Order_Date,  
    p.Product_Name,  
    od.Quantity,  
    od.Unit_Price,  
    od.Subtotal,  
    o.Total_Amount  
FROM Customers c  
JOIN Orders o  
    ON c.Customer_ID = o.Customer_ID  
JOIN Order_Details od  
    ON o.Order_ID = od.Order_ID  
JOIN Products p  
    ON od.Product_ID = p.Product_ID  
ORDER BY o.Order_Date DESC;
```

This query combines information from four tables using JOIN.

16. Order History of a Specific Customer

To find the complete order history of Rahul Kumar:

```
SELECT
    c.Customer_Name,
    o.Order_ID,
    o.Order_Date,
    p.Product_Name,
    od.Quantity,
    od.Subtotal,
    o.Total_Amount
FROM Customers c
JOIN Orders o
    ON c.Customer_ID = o.Customer_ID
JOIN Order_Details od
    ON o.Order_ID = od.Order_ID
JOIN Products p
    ON od.Product_ID = p.Product_ID
WHERE c.Customer_ID = 1
ORDER BY o.Order_Date DESC;
```

17. Calculate Total Amount Using SQL

Instead of manually calculating the total amount, SQL can calculate it using SUM().

```
SELECT
    o.Order_ID,
    c.Customer_Name,
    SUM(od.Quantity * od.Unit_Price) AS Calculated_Total
FROM Orders o
JOIN Customers c
    ON o.Customer_ID = c.Customer_ID
JOIN Order_Details od
```

```
ON o.Order_ID = od.Order_ID
GROUP BY o.Order_ID, c.Customer_Name;
```

18. Find Total Orders of Each Customer

```
SELECT
    c.Customer_ID,
    c.Customer_Name,
    COUNT(o.Order_ID) AS Total_Orders
FROM Customers c
LEFT JOIN Orders o
    ON c.Customer_ID = o.Customer_ID
GROUP BY c.Customer_ID, c.Customer_Name;
```

This query displays the number of orders placed by each customer.

19. Find Total Amount Spent by Each Customer

```
SELECT
    c.Customer_ID,
    c.Customer_Name,
    SUM(o.Total_Amount) AS Total_Spent
FROM Customers c
JOIN Orders o
    ON c.Customer_ID = o.Customer_ID
GROUP BY c.Customer_ID, c.Customer_Name;
```

This report helps identify how much each customer has spent.

20. Find Products Ordered in Large Quantities

```
SELECT
    p.Product_Name,
    SUM(od.Quantity) AS Total_Quantity_Ordered
FROM Products p
JOIN Order_Details od
```

ON p.Product_ID = od.Product_ID

GROUP BY p.Product_ID, p.Product_Name

ORDER BY Total_Quantity_Ordered DESC;

This query can be used to identify the most frequently ordered products.

21. Advantages of the Order Management System

The proposed system provides several advantages:

1. It reduces manual order management.
 2. It maintains accurate customer order records.
 3. It stores complete product and order information.
 4. It makes order modification easier.
 5. It reduces data duplication through relational database design.
 6. It allows quick generation of customer order history.
 7. It helps calculate order totals accurately.
 8. It provides useful reports for business analysis.
 9. Primary keys and foreign keys maintain data integrity.
 10. SQL queries allow easy searching and reporting.
-

22. Conclusion

The Order Management System demonstrates how a relational database can be used to manage customer orders efficiently. The Orders table stores the main information about each order, while the Order_Details table stores the individual products and quantities associated with each order.

Using SQL, we can insert new orders, modify existing orders, update quantities and prices, calculate total amounts, and generate customer order history reports. The relationships between the tables ensure that customer, product, and order information remains organized and consistent.

Therefore, this project provides a practical implementation of important DBMS concepts such as table creation, primary keys, foreign keys, data insertion, data modification, joins, aggregate functions, and report generation.

Final Outcome

The completed Order Management System successfully fulfills the required tasks:

- ✓ Orders and Order_Details tables designed and created.
- ✓ Customer product orders managed.
- ✓ Order date, quantity, and total amount stored.

- ✓ Order insertion operations performed.
- ✓ Order modification operations performed.
- ✓ Customer order history reports generated.
- ✓ SQL joins and aggregate functions demonstrated.
- ✓ Relational database concepts implemented.